

MÓDULO 1 • PIPELINE DE DATOS Y PREPARACIÓN PROFESIONAL

Selección de Características y Tratamiento del Desbalanceo

Material teórico — Feature Selection, métricas y SMOTE

Objetivo del material

Comprender cómo seleccionar las características más útiles para un modelo de Machine Learning y cómo abordar problemas de desbalanceo de clases en tareas de clasificación.

1. ¿Qué es Feature Selection?

La **selección de características** o **Feature Selection** consiste en elegir, de un conjunto de variables disponibles, aquellas que aportan información relevante para el problema.

Es diferente de Feature Engineering. En Feature Engineering podemos crear nuevas variables; en Feature Selection decidimos cuáles conservar.

2. ¿Por qué seleccionar características?

- Reducir características irrelevantes.
- Eliminar información redundante.
- Reducir la complejidad del modelo.
- Disminuir el tiempo de entrenamiento.
- Facilitar la interpretación.
- Reducir el riesgo de sobreajuste en algunos escenarios.

3. Características relevantes, redundantes e irrelevantes

Relevante: aporta información útil para predecir la variable objetivo.

Irrelevante: aporta poca o ninguna información útil para la tarea.

Redundante: contiene información muy similar a otras características.

4. El problema de utilizar demasiadas características

Agregar muchas variables no garantiza un modelo mejor. Algunas pueden contener ruido, duplicar información o aumentar innecesariamente la complejidad.

Principio importante

La selección de características debe estar guiada por el problema, los datos y la evaluación experimental, no simplemente por conservar la mayor cantidad posible de columnas.

5. Métodos de selección

Método	Tipo	Idea principal
Correlación	Filter	Detecta relaciones entre variables numéricas.
VarianceThreshold	Filter	Elimina variables con muy poca variabilidad.
SelectKBest	Filter	Selecciona las K características con mejor puntuación.
Mutual Information	Filter	Mide dependencia entre características y objetivo.
RFE	Wrapper	Elimina características de forma iterativa utilizando un modelo.
L1 / Lasso	Embedded	Puede llevar algunos coeficientes a cero.
Random Forest	Embedded	Permite estimar importancia de características.

6. Métodos Filter

Los métodos **Filter** analizan las características mediante medidas estadísticas antes de entrenar el modelo.

- Son relativamente rápidos.
- Pueden aplicarse como etapa previa al modelado.
- No dependen necesariamente de un algoritmo específico.
- Deben interpretarse según el tipo de problema.

7. Selección mediante correlación

La correlación permite estudiar la relación entre variables numéricas. Si dos características presentan una correlación muy alta, puede existir redundancia.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```

Una correlación alta entre dos variables no significa automáticamente que una deba eliminarse. La decisión debe considerar el problema y el modelo.

8. VarianceThreshold

VarianceThreshold elimina características cuya variabilidad es inferior a un umbral determinado.

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.0)
X_new = selector.fit_transform(X)
```

Una característica constante tiene varianza cero y, en general, no permite distinguir entre observaciones.

9. SelectKBest

SelectKBest permite seleccionar las K características con mejores puntuaciones según una función estadística.

```
from sklearn.feature_selection import SelectKBest, f_classif
```

```
selector = SelectKBest(score_func=f_classif, k=5)
X_new = selector.fit_transform(X, y)
```

La función estadística debe ser adecuada al tipo de problema y a las características utilizadas.

10. Mutual Information

La información mutua permite estimar la dependencia entre una característica y la variable objetivo.

```
from sklearn.feature_selection import mutual_info_classif

scores = mutual_info_classif(X, y, random_state=42)
```

Un valor mayor puede indicar una relación informativa, aunque no implica necesariamente una relación causal.

11. Métodos Wrapper

Los métodos Wrapper utilizan un modelo para evaluar diferentes conjuntos de características.

Una técnica conocida es **RFE**, Recursive Feature Elimination.

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
selector = RFE(model, n_features_to_select=5)
X_new = selector.fit_transform(X, y)
```

12. Métodos Embedded

Los métodos Embedded realizan la selección como parte del entrenamiento del modelo.

Dos ejemplos frecuentes son:

- Regularización L1.
- Importancia de características en modelos basados en árboles.

13. L1 y Lasso

La regularización L1 puede llevar algunos coeficientes del modelo a cero. Esto permite identificar características que el modelo no está utilizando de la misma manera.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    penalty='l1',
    solver='liblinear'
)
```

14. Importancia de características con árboles

Modelos como Random Forest pueden proporcionar estimaciones de importancia de características.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)
model.fit(X, y)

importancias = model.feature_importances_
```

Estas importancias deben interpretarse con cuidado y no deben considerarse automáticamente como una prueba de causalidad.

15. Selección y Cross-Validation

Si seleccionamos características utilizando todo el dataset antes de realizar Cross-Validation, podemos introducir información del conjunto de validación en el proceso.

Regla:

Las operaciones que aprenden información de los datos deben realizarse dentro del proceso de entrenamiento y validación, no utilizando anticipadamente todo el dataset.

16. Data Leakage en Feature Selection

Existe Data Leakage cuando una transformación utiliza información que no debería estar disponible durante el entrenamiento.

Por ejemplo, calcular estadísticas sobre todo el dataset y luego dividirlo puede contaminar el conjunto de prueba.

17. ¿Qué es el desbalanceo de clases?

En un problema de clasificación existe desbalanceo cuando las clases tienen cantidades de observaciones muy diferentes.

Por ejemplo:

```
Clase 0: 9500 observaciones
Clase 1: 500 observaciones
```

La clase 1 representa solamente el 5 % de los datos.

18. ¿Por qué el desbalanceo es un problema?

Un modelo puede aprender a favorecer la clase mayoritaria y aun así obtener una Accuracy aparentemente alta.

Ejemplo

Si el 95 % de los clientes no abandona y el modelo predice siempre 'no abandona', podría obtener aproximadamente 95 % de Accuracy y, sin embargo, no detectar ningún abandono.

19. Matriz de confusión

La matriz de confusión permite analizar las predicciones comparando las clases reales con las predichas.

	Predicción positiva	Predicción negativa
Real positiva	Verdadero Positivo (TP)	Falso Negativo (FN)
Real negativa	Falso Positivo (FP)	Verdadero Negativo (TN)

20. Precision, Recall y F1-score

Precision: de las predicciones positivas realizadas, ¿cuántas fueron realmente positivas?

Recall: de todas las observaciones realmente positivas, ¿cuántas logró detectar el modelo?

F1-score: combina Precision y Recall mediante su media armónica.

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

21. Estrategias para tratar el desbalanceo

- Modificar los pesos de las clases.
- Oversampling de la clase minoritaria.
- Undersampling de la clase mayoritaria.
- SMOTE.
- Utilizar métricas adecuadas.
- Comparar diferentes estrategias.

22. Class Weight

Algunos modelos permiten asignar mayor peso a las clases minoritarias.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    class_weight='balanced',
    max_iter=1000
)
```

Esta estrategia no crea nuevas observaciones. Modifica la importancia que tienen las clases durante el entrenamiento.

23. Oversampling y Undersampling

Oversampling: aumenta la representación de la clase minoritaria.

Undersampling: reduce la representación de la clase mayoritaria.

Ambas estrategias deben aplicarse solamente sobre los datos de entrenamiento.

24. ¿Qué es SMOTE?

SMOTE significa **Synthetic Minority Over-sampling Technique**.

En lugar de simplemente duplicar observaciones de la clase minoritaria, SMOTE genera nuevos ejemplos sintéticos a partir de observaciones existentes y sus vecinos.

Idea conceptual

SMOTE busca crear nuevos puntos dentro de regiones donde existen observaciones de la clase minoritaria.

25. Aplicar SMOTE

SMOTE forma parte de la biblioteca **imbalanced-learn**.

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
```

Después de la transformación es posible comprobar la nueva distribución de las clases.

26. SMOTE y Data Leakage

Uno de los errores más importantes es aplicar SMOTE antes de dividir los datos en entrenamiento y prueba.

Incorrecto:

```
X_resampled, y_resampled = smote.fit_resample(X, y)
X_train, X_test, y_train, y_test = train_test_split(...)
```

Correcto:

```
X_train, X_test, y_train, y_test = train_test_split(...)

X_train, y_train = smote.fit_resample(X_train, y_train)
```

27. Pipeline con SMOTE

Cuando se utiliza Cross-Validation, una alternativa profesional es incorporar SMOTE dentro de un Pipeline de imbalanced-learn.

```
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('model', LogisticRegression(max_iter=1000))
])
```

De esta manera, el sobremuestreo se realiza dentro del proceso de entrenamiento de cada partición.

28. Selección + Balanceo

En un proyecto real pueden combinarse diferentes etapas:

- Imputación.
- Feature Engineering.
- Feature Selection.
- Escalado.
- SMOTE.
- Modelo.
- Evaluación.

El orden exacto depende de las características del problema y debe diseñarse cuidadosamente para evitar Data Leakage.

29. Flujo profesional completo

Un flujo simplificado puede representarse como:

```
Dataset
↓
EDA
↓
Limpieza
↓
Feature Engineering
↓
Separación Train/Test
↓
Feature Selection
↓
Balanceo del entrenamiento
↓
Modelo
↓
Evaluación
```

30. Errores frecuentes

- Eliminar características únicamente porque tienen correlación baja.
- Mantener todas las variables sin evaluar su utilidad.
- Seleccionar características utilizando el conjunto de prueba.
- Aplicar SMOTE antes del train/test split.
- Utilizar Accuracy como única métrica en datasets desbalanceados.
- No analizar la matriz de confusión.
- Aplicar SMOTE sobre datos de validación.
- No comparar diferentes estrategias.
- No documentar las decisiones.

31. Buenas prácticas

- Comprender el problema antes de seleccionar variables.
- Separar entrenamiento y prueba correctamente.
- Realizar las transformaciones dentro de Pipelines cuando corresponda.
- Utilizar Cross-Validation.
- Elegir métricas relacionadas con el objetivo del proyecto.
- Comparar modelos y estrategias.
- Documentar las características seleccionadas.
- Verificar que no exista Data Leakage.

32. Caso práctico

Supongamos un sistema que debe detectar clientes con riesgo de abandonar un servicio.

El dataset contiene 10 000 clientes:

```
Clase 0 – permanece: 9 000
Clase 1 – abandona: 1 000
```

El proyecto podría seguir estos pasos:

1. Explorar la distribución de las clases.
2. Analizar las características.
3. Eliminar variables claramente irrelevantes.
4. Separar entrenamiento y prueba.
5. Seleccionar características dentro del proceso de entrenamiento.
6. Aplicar SMOTE solamente sobre entrenamiento.
7. Entrenar el modelo.
8. Evaluar con Precision, Recall y F1-score.
9. Comparar con una estrategia sin SMOTE.
10. Analizar si el modelo cumple el objetivo del negocio.

33. Idea central

Seleccionar características y tratar el desbalanceo son decisiones del proceso de preparación, no pasos aislados.

El objetivo no es modificar los datos de cualquier manera, sino construir un conjunto de entrenamiento representativo y un flujo reproducible que permita evaluar correctamente el modelo.

34. Glosario

Concepto	Definición
Feature Selection	Proceso de seleccionar características relevantes para un modelo.
Filter Method	Método de selección basado en criterios estadísticos.
Wrapper Method	Método que utiliza un modelo para evaluar subconjuntos de características.
Embedded Method	Método donde la selección forma parte del entrenamiento.
VarianceThreshold	Método que elimina variables con variabilidad insuficiente.
SelectKBest	Selecciona las K características con mejor puntuación estadística.
RFE	Eliminación recursiva de características.
Desbalanceo	Situación donde las clases tienen cantidades muy diferentes de observaciones.
Oversampling	Aumento de la representación de la clase minoritaria.
Undersampling	Reducción de la representación de la clase mayoritaria.
SMOTE	Técnica que genera ejemplos sintéticos de la clase minoritaria.
Class Weight	Asignación de diferentes pesos a las clases durante el entrenamiento.

Concepto	Definición
Data Leakage	Uso indebido de información que no debería estar disponible durante el entrenamiento.